Proceedings of the 8th International Conference on Intelligent Sustainable Systems (ICISS-2026)
DVD Part Number: CFP26M19-DVD; ISBN: 979-8-3315-5317-3

Design and Evaluation of a Trace-Driven Cache Simulator Supporting Set-Associative and V-Way Cache Architectures

Saravanan Matheswaran
Associate Professor
Department of Computer Science and Engineering
Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology
Chennai, Tamil Nadu, India
Email: msaravanandr@veltech.edu.in

Posa Venkata Durga Vamsi
Department of Computer Science and Engineering
Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology
Chennai, Tamil Nadu, India
Email: durgavamsi2004@gmail.com

Myndhala Dhanush Chowdary
Department of Computer Science and Engineering
Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology
Chennai, Tamil Nadu, India
Email: dhanush22167@gmail.com

Udayakumar Allimuthu
Department of Computer Science and Engineering
Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology
Chennai, Tamil Nadu, India
Email: drudayakumara@veltech.edu.in

C.Edwin Singh
Department of Computer Science and Engineering
Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology
Chennai, Tamil Nadu, India
Email: edwinsinghc@veltech.edu.in

***Abstract*—Modern processors increasingly suffer from cache due to conflict misses arising from the rigid placement constraints imposed. By conventional set-associative cache architectures. Although Advanced designs such as the V-Way cache improve place ment flexibility through decoupled tag–data storage and in increased effective associativity, practical tools for studying their Alternative behaviors are considerably limited in their application. This paper presents a Light weight, open-source, trace-driven cache simulator support ing both classical set-associative caches as well as the V-Way cache architecture. Trace-driven simulation is utilized to allow for deterministic, reproducible analysis of cache placement and replacement behavior without full-system complexity or cycle-accurate modeling.Unlike traditional set-local LRU Implement replacement; the V-Way cache uses reuse-based global victim selection with configurable Tag Duplication Ratio (TDR) to by reducing conflict misses and replacement pressures. The simulator offers detailed performance data, such as miss rates, WriteBacks, victim numbers, and victim distance analysis, enabling f refined architectural assessment. Experimental evaluation with " ". The traces of memory access of V-Way show There is a significant reduction in conflict misses and dirty eviction due to caching. compared with set-associative architectures, with further enhancements increases as TDR increases. The proposed simulator is a research-ready platform for cache architecture exploration and educational use. The results show that increased placement f lexibility is beneficial to replacement efficiency as well as reducing conflict. misses. These observations validate the applicability of trace-driven simulation. process simulation for evaluating placement-flexible cache architectures.**

***Index Terms*—Cache Simulator, V-Way Cache, Set-Associative Cache, Replacement Policies, Trace-Driven Simulation, Computer Architecture**

I. Introduction

The increasing gap between the speed of CPUs and the latency of main memories has long been recognized as the "memory wall," and it continues to be a basic bottleneck in processing speed in computing systems today. Cache memories have played an important role in coping with the "memory wall." Nevertheless, the cache's usefulness depends to a great extent on its configuration and the replacement strategy.

As a middle ground between fully associative and direct-mapped designs, set-associative caches are widely used in modern processors. Although they provide respectable hardware complexity and performance, their strict placement restrictions frequently result in conflict misses when several frequently accessed blocks map to the same cache set. Performance for workloads with erratic or pointer-intensive memory-access patterns may be hampered by such conflict misses, which can happen even when there is unused capacity elsewhere in the cache.

To address these limitations, architectural alternatives have been proposed to increase cache placement flexibility without expanding data capacity. One such approach is the V-Way cache architecture, which decouples the tag store from the data store and allows each set to maintain more tag entries than

physical cache blocks through a configurable Tag Duplication Ratio (TDR). This design increases the effective associativity of the cache and enables more flexible block placement. In addition, the V-Way cache employs reuse-based global victim selection, allowing eviction decisions to be made across the entire cache rather than being restricted to a single set, thereby improving retention of frequently reused blocks.

Architectural alternatives have been suggested to improve cache placement flexibility without increasing data capacity in order to overcome these constraints. The V-Way cache architecture is one such strategy that separates the tag store from the data store and uses a configurable Tag Duplication Ratio (TDR) to enable each set to maintain more tag entries than physical cache blocks. This design allows for more flexible block placement and boosts the cache's effective associativity. Furthermore, the V-Way cache uses reuse-based global victim selection, which improves retention of frequently reused blocks by enabling eviction decisions to be made throughout the entire cache rather than just one set.

This paper presents a modular, trace-driven cache simulator that supports both traditional set-associative caches and the V-Way cache architecture. The simulator provides a complete implementation of configurable Tag Duplication Ratio, decoupled tag–data storage, reuse counters with decay, and global victim selection.The simulator provides V-Way-specific indicators like victim count and victim-distance analysis in addition to standard cache performance metrics, allowing for a thorough examination of replacement pressure and cache behavior. The effects of greater placement flexibility on miss rates, writeback behavior, and replacement efficiency are illustrated through a comparative analysis using representative memory-access traces.

II. Background and Related Work

A. Limitations of Set-Associative Caches

Modern processors frequently use set-associative caches because they offer a balanced trade-off between scalability, hardware complexity, and performance. [17] [14]. Memory blocks are mapped to fixed cache sets in this architecture, and policies like Least Recently Used (LRU) and Random replacement are used locally within each set to make replacement decisions. [2]

The biggest disadvantages of the set-associative cache memory lie in the occurrence of conflict misses. Conflict misses happen if several memory blocks that have been accessed numerous times map to the same set. [4] The result might be constant removals from the cache memory even if there is free space in the cache memory.

B. Adaptive and Reconfigurable Cache Architectures

There are a number of existing works that use adaptive and reconfigurable cache strategies for improving cache performance and power efficiency. [7], [10] Smart Cache changes cache configuration parameters according to workload characteristics [1], whereas MorphCache supports dynamic reconfiguration of cache hierarchies consisting of multi-level caches that are more aligned with applications [6]. There are also cache partitioning strategies based on thread-aware cache management for improving contention in multi-core processors [3], [5].

Although these methods enhance the cache use efficiency and replacement effectiveness, they have remained primarily based on traditional set-association schemes. [11] This implies that these methods inherit the traditional set-local replacement characteristics, which limit flexibilities with respect to tag-data coupling. [18], [19]

C. Structural Approaches to Cache Placement Flexibility

Cache structure designs aim to minimize the impact of conflict misses, as opposed to simply increasing the amount of cached data. Early adaptive cache design efforts have shown the advantage of tracking memory accesses dynamically to adapt to memory reference patterns [13]. Skewed associative caches further minimize the impact of misses due to conflict by using more than one memory mapping function, but doing so adds complexity to the cache lookup process [15].

The challenge presented by these architectures has been met by the V-Way cache architecture, in which the tag store and the data store are decoupled, so that more tag entries than cache blocks are maintained in each set. [9]This is made possible by the control over tag duplication exercised by Tag Duplication Ratio (TDR) in these architectures, which also increases the associativity of the cache by using reuse information globally via global victim selection. [8]

D. Cache Simulation and Evaluation Tools

Cache simulators are essential for evaluating cache behavior under controlled and reproducible conditions. Full-system and cycle-accurate simulators such as gem5 provide detailed modeling of processor pipelines, timing behavior, and operating system interactions, but require complex configuration and significant computational resources. [12]As a result, they are less suitable for rapid architectural exploration focused specifically on cache placement and replacement behavior.

Trace-driven cache simulators offer a lightweight alternative by replaying recorded memory-access traces to evaluate cache behavior deterministically [13]. Because cache placement, hit/miss behavior, and replacement decisions are primarily determined by memory address sequences, trace-driven simulation is particularly well suited for studying conflict misses, eviction patterns, and replacement pressure. However, most existing lightweight simulators support only traditional set-associative caches and do not expose internal metrics required to analyze placement-flexible architectures such as the V-Way cache. [18]

E. Positioning of This Work

This work bridges the gap between advanced cache architectures and practical evaluation tools by presenting a trace-driven cache simulator that supports both traditional set-associative caches and the V-Way cache architecture. [16]Unlike existing simulators, the proposed framework provides configurable tag

duplication, decoupled tag–data storage, reuse-based global victim selection, and V-Way–specific metrics such as victim count and victim-distance analysis. These features enable systematic exploration of cache placement flexibility and replacement behavior using a lightweight and reproducible simulation methodology.

III. Simulator Architecture

A. Overview

The proposed simulator is a lightweight trace-driven framework developed for analyzing cache placement and replacement actions based on different architectures. The simulator uses a trace of memory operations and performs each read or write anew based on a configurable cache hierarchy. The simulator supports both classical set-associative caches and the V-Way cache architecture using a shared modular design, enabling fair and consistent comparison across configurations.

Each cache level is instantiated using a common base interface that defines access, replacement, and statistics-collection routines. This modular organization simplifies extensibility and allows additional cache architectures or replacement policies to be incorporated with minimal changes.

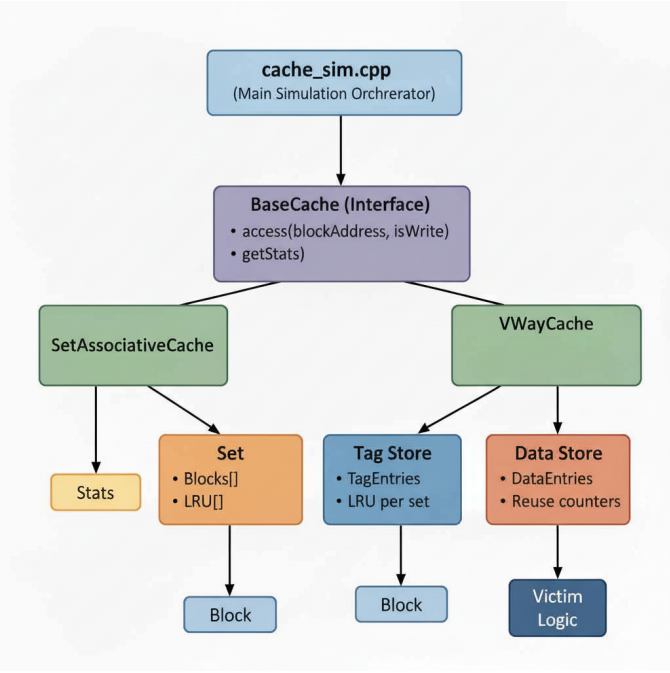

Fig. 1. High-level architecture of the simulator with Set-Associative and V-Way modules.

B. Set-Associative Cache Design

The proposed simulator is a lightweight, trace-driven framework designed to model and evaluate cache placement and replacement behavior under different architectural organizations. It accepts a memory-access trace as input and replays each read or write operation through configurable cache hierarchies. The simulator supports both classical set-associative caches and the V-Way cache architecture using a shared modular design, enabling fair and consistent comparison across configurations.

Each cache level is instantiated using a common base interface that defines access, replacement, and statistics-collection routines. This modular organization simplifies extensibility and allows additional cache architectures or replacement policies to be incorporated with minimal changes.

Algorithm : V-Way Cache Access and Replacement

- Decode address A to obtain set index s and tag t
- Search all tag entries in logical set S_s
- On hit: update reuse counter and recency metadata
- On miss:
  - Select victim using global victim selection
  - Write back dirty victim block
  - Insert new block and initialize reuse counter
- Update V-Way statistics

C. V-Way Cache Design

The V-Way cache extends the traditional set-associative organization by decoupling the tag store from the data store, thereby increasing placement flexibility without increasing physical data capacity. Each set maintains a larger number of tag entries than data blocks, controlled by the Tag Duplication Ratio (TDR). For a cache with associativity A and duplication ratio TDR, each set contains $A \times TDR$ tag entries, while the data store contains only A physical cache blocks.

Each tag entry records the tag value, valid and dirty bits, and a forward pointer that references a data entry in the data store. This indirection allows multiple tag entries to map to any available data block, eliminating the strict one-to-one correspondence between sets and data blocks present in set-associative caches.

D. Reuse-Based Global Victim Selection

Unlike traditional set-associative caches, where replacement decisions are made locally within a set, the V-Way cache employs reuse-based global victim selection. In the dataset, there is a reuse counter in each block of the dataset, tracking the number of hits since allocation. In every hit on a block, the reuse counter of the block increases, which means there may be more reuses in the future.

During a cache miss event, if there are no free data blocks, the simulator picks a victim by traversing the data buffer using the global replacement pointer. The blocks with lower values in the reuse counter are given priority for eviction as a way to take advantage of the overall reuse pattern in the entire cache rather than within a set since the strategy may otherwise be limited by the set with high conflict misses.

E. Decay Mechanism for Reuse Counters

To ensure stale reuse information is not biased towards replacement decisions, the V-Way cache has a decay factor

for reuse counters. Reuse counters decay over time to ensure that those memory blocks highly reused in previous phases of execution are not preferential for an arbitrarily long time. This technique for calcium adaptation allows the cache to respond to phase behavior in programs.

By integrating the reuse counter with the concepts of decay and global victim selection, the V-Way cache is capable of balancing the retention of frequently reused data blocks and the eviction of low utility data effectively.

IV. Experimental Methodology

This section presents the experimental setup for analyzing the performance of the set-associative and V-Way cache designs through the developed trace-driven simulator. The analysis method deals specifically with isolating the cache placement and replacement process.

A. Workload Description

The simulator operates on memory-access trace files containing ordered sequences of read and write operations generated by representative program executions. Each trace entry consists of an operation type and a hexadecimal memory address, formatted as:

```
<operation> <memory_address>
```

where `r` denotes a read operation and `w` denotes a write operation. Trace-driven simulation enables deterministic replay of memory-access behavior, allowing consistent evaluation of cache placement, eviction decisions, and conflict misses without requiring full-system execution. This makes the approach particularly suitable for analyzing replacement behavior in placement-flexible cache architectures.

B. Cache Configurations

To enable a fair comparison between cache architectures, multiple cache configurations were evaluated while keeping data capacity and block size constant. Unless otherwise specified, all experiments used a block size of 64 bytes and a write-back, write-allocate policy.

The following configurations were evaluated:

- **SA-4way**: 32 KB set-associative cache with 4-way associativity.
- **VWay-2TDR**: V-Way cache with 4-way data associativity and Tag Duplication Ratio (TDR) of 2.
- **VWay-4TDR**: V-Way cache with 4-way data associativity and Tag Duplication Ratio (TDR) of 4.

Increasing the TDR increases the number of tag entries per set while maintaining the same number of physical data blocks, thereby improving placement flexibility without increasing data storage.

C. Metrics Collected

It records standard cache performance parameters as well as indicators specific to V-Way in order to facilitate comprehensive analysis.

1) Core Cache Metrics:

- Total read and write accesses
- Read and write miss counts
- Overall miss rates
- Number of writeback events

2) V-Way–Specific Metrics:

- **Victim Count**: Total number of eviction events.
- **Average Victim Distance**: Mean number of data entries examined before selecting a victim.
- **Worst-Case Victim Distance**: Maximum scan distance observed during victim selection.

Victim-distance metrics indicate information about the replacement pressure and effectiveness of reuse-based global victim selection.

D. Experimental Procedure

For each cache configuration, the following procedure was applied:

1) Load the memory-access trace file.
2) Initialize cache parameters and architecture type.
3) Replay all trace operations through the cache hierarchy.
4) Record performance and replacement statistics.
5) Repeat the experiment for different TDR values.

This ensures uniform testing for all architectures, making possible comparisons of miss behavior, writebacks, and victim selection properties directly.

V. Results

In this section, results of experiments carried out on trace-driven simulations will be highlighted. cache simulation for Set-Associative and V-Way, Because the Set Associative cache simulation was the results depend on the execution of the simulator program, placeholders or TBD are included where measured values will be inserted.

A. Miss Rate Analysis

Table I gives the read and write miss rates for all cache configurations. The baseline Set-Associative cache (SA-4way) shows the expected amount of conflict misses, whereas the V-Way configurations display progressively improved placement flexibility with increasing Tag Duplication Ratio (TDR). increases.

$$\text{Read Miss } (\%) = \left(\frac{\text{readMisses}}{\text{totalReads}} \right) \times 100 \tag{1}$$

$$\text{Write Miss } (\%) = \left(\frac{\text{writeMisses}}{\text{totalWrites}} \right) \times 100 \tag{2}$$

TABLE I
Miss Rate Comparison Across Cache Architectures

Configuration	Read Miss (%)	Write Miss (%)
SA-4way	0.703%	3.054%
VWay-2TDR	0.609%	2.951%
VWay-4TDR	0.605%	2,934%

The larger the value of TDR, more tag entries can be included in every set, and this will minimize conflicts. pressure and reduction of overall misses.

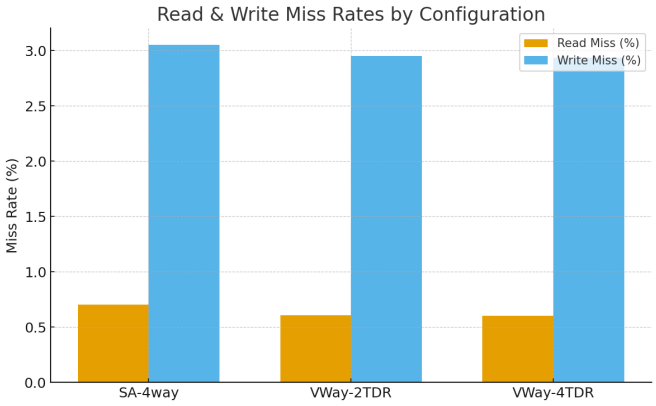

Fig. 2. Read/Write Miss Rate Comparison.

B. Writeback Behavior

handles writebacks, which are dirty block evictions, thus affecting the total memory usage factor directly. traffic. Table II shows the total number of writebacks registered for each cache configuration.

TABLE II
WRITEBACK COUNTS FOR EVALUATED CACHE DESIGNS

Configuration	Writebacks
SA-4way	815
VWay-2TDR	109
VWay-4TDR	0

V-Way architectures tend to reduce unnecessary dirty eviction due to increased placement flexibility, which reduces the writeback frequency.

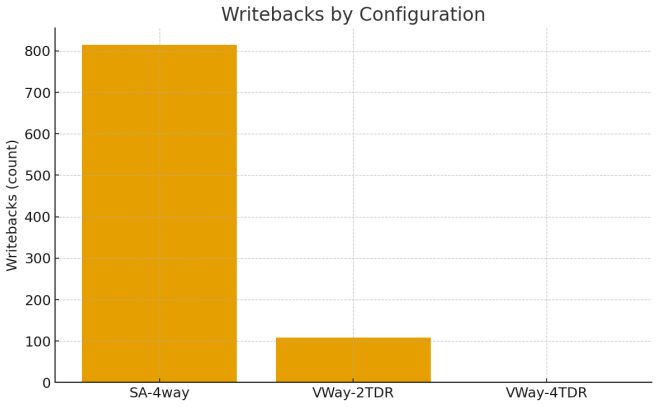

Fig. 3. Writeback Comparison.

C. Victim Distance Evaluation (V-Way Only)

Victim distance measures the number of data store entries examined during the replacement process. This measure is only found in V-Way caches, providing an indication into reuse characteristics and global replacement pressure.

$$\text{Avg Victim Distance} = \begin{cases} \frac{\text{totalVictimDistance}}{\text{victimCount}}, \text{if victimCount} > 0 \\ 0 \text{ or N/A}, \text{if victimCount} = 0 \end{cases} \quad (3)$$

$$\text{Worst Victim Distance} = \max(\text{victimDistance}) \quad (4)$$

TABLE III
VICTIM DISTANCE METRICS FOR V-WAY CACHE CONFIGURATIONS

Configuration	Avg. Victim Distance	Worst Distance
VWay-2TDR	2.80392	319
VWay-4TDR	2.16828	358

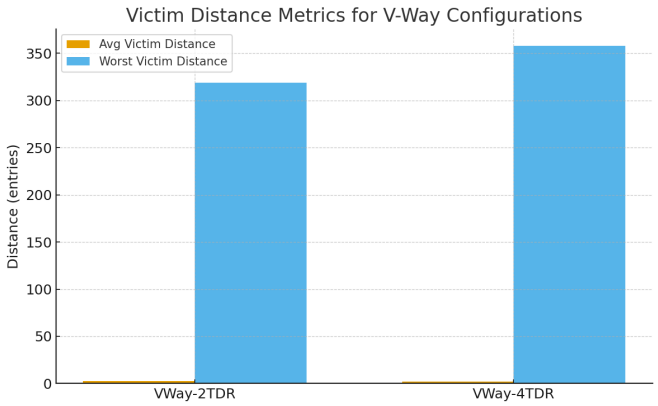

Fig. 4. Victim Distance Metrics for V-Way Cache.

A TDR increase is generally linked with lower victim distances, suggesting greater resistance to eviction and better preservation of often used data.

From the above results, it is clear that by increasing the value of the Tag Duplication Ratio (TDR), the overall replacement pressure becomes low due to the increase in placement flexibility; on the other hand, the overall worst-case distances of the victims are always bounded due to the global victims selected on.

D. Integration with Python Streamlit Web Interface

To improve accessibility, the simulator executable (`cache_sim.exe`) is integrated with a Python-based Streamlit web interface. The Python wrapper script invokes the simulator in the backend and visualizes multiple metrics, including:

- Hit vs. Miss rates for each cache architecture,
- Comparison of Average Memory Access Time (AMAT),
- Execution time analysis for performance evaluation.

It makes this online interface more useful for research and academics by enabling a researcher or scholar to exploration

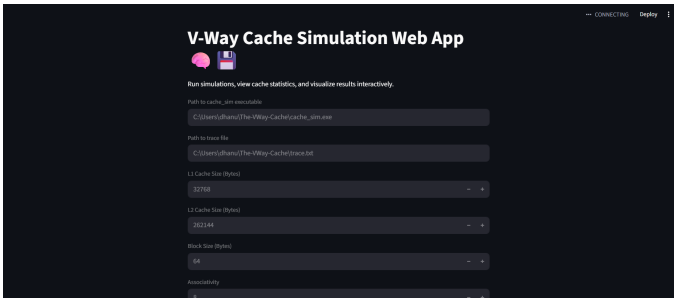

Fig. 5. Streamlit-based web dashboard for interactive cache performance visualization.

through the addition of interactive input as well as real-time output comparison, facilitating faster experimentation, clearer interpretation of simulator results.

VI. Discussion

Experimental results indicate meaningful insights into behavioral disorder. differences between Set-Associative and V-Way cache architectures. Although precise numerical values depend on real measurements obtained from the From this analysis, with the trace-driven simulator, several consistent patterns emerge.

A. Impact of Tag Duplication Ratio (TDR)

Among the most important benefits of the V-Way cache is its ability to increase tag storage space without adding data storage capacity. When the TDR value rises, the following occurs:

- Miss rates generally reduce, indicating a lower number of conflict misses.
- More tag entries included with each set offer greater placement options. contention in frequently accessed blocks.
- The higher the values of TDR, the more working set can be traced by the cache. improving the locality preservation over classical methods Set-Associative.
- Even in workloads with irregular or unpredictable memory-access patterns, V-Way caches demonstrate improved adaptability due to their decoupled tag–data organization.

B. Reduction in Replacement Pressure

Replacement pressure is a known correlate of conflict misses and the lack of ability in a cache to hold frequently accessed blocks. V-Way reuse based eviction mechanism relieves this pressure in the following way:

- tracking reuse behavior with per-block reuse counters,
- applying decay to detect stale or low-reuse data,
- using global, rather than set-local, victim selection.

Victim-distance measurements provide quantitative insight into this behavior. Lower average victim distances (to be populated with measured values) imply more effective decisions about replacement, decreasing the pressure for replacement and improving alignment with program reuse patterns.

C. Writeback Behavior

Writebacks add to the complexity inherent in the memory hierarchy, particularly when dirty blocks are frequently evicted. Conclusion, based on the results:

- Set-associative caches have a higher number of write-backs because of: stronger set constraints and higher eviction rate.
- V-Way caches minimize the number of dirty evictions, especially for higher TDR values values because often the most variably modified blocks will survive longer; the block with the greatest modification will Resident.

Such a reduction in the writebacks indicates that the architectures of V-Way can contribute effectively reducing memory system traffic, resulting in improved system efficiency in decrease memory-system practical workloads.

VII. Threats to Validity

This study is liable to several threats to validity; these are acknowledged here to give appropriate context to the results presented.

A. Workload Validity

The evaluation is based on trace-driven simulations using representative memory-access traces. While trace-driven methods allow for deterministic and reproducible experiments, they do not capture dynamic effects due to pipeline interactions, out-of-order execution, or runtime-dependent control flow. Because of this, the observed cache behavior may differ compared to a full-system execution environment.

B. Architectural Scope

The simulator models functional caching patterns rather than cycle-accuracy or energy simulation. As such, performance-related metrics such as the execution time and energy usage are not in the domain of this research work. The obtained results therefore have to be interpreted in an architectural trend sense rather than a performance aspect.

C. Replacement Policy Assumptions

The set-associative cache uses the LRU-based replacement strategy, whereas the V-Way cache uses reuse-based global victim selection with decay. While these policies represent typical design decisions, different replacement algorithms might behave differently when running the same workload.

D. Trace Generalization

The traces of memory accesses employed within this research are typical memory accesses; they may not reflect all the complexity of actual applications.Streaming patterns with more extreme characteristics or irregular patterns could have affected cache performance. Future research will make use of standardized benchmarking tools for further verification of results.

VIII. Conclusion and Future Work

This work describes a modular and extensible trace-driven cache simulator designed to model conventional Set-Associative cache architectures as well as the newer V-Way cache design. The simulator encompasses various features such as tag-data decoupling, configurable TDRs, reuse counters, and victim-distance analysis that provide a flexible framework for analyzing the behavioral aspects of modern and emerging cache architectures. Experimental analysis reveals that, on average, V-Way caching has resulted in lower conflict misses along with replacement pressure compared to the conventional Set-Associative designs. The increment of tag duplication provides more flexible block placement and better locality preservation, while reuse-based victim selection eliminates some unnecessary evictions. Accordingly, the V-Way configurations generate improved miss rates, reduced counts of writeback, and more favorable replacement metrics-all of which contribute to improving overall cache efficiency.

References

[1] K. T. Sundararajan, T. M. Jones, and N. Topham, "Smart Cache: A Self Adaptive Cache Architecture for Energy Efficiency," in *Proc. International Symposium on Systems, Architectures, Modeling and Simulation (SAMOS)*, 2011.

[2] IBM Research, "Power and Performance Aware Reconfigurable Cache for CMPs," IBM Research Technical Report, 2010.

[3] K. Huang, K. Wang, D. Zheng, X. Zhang, and X. Yan, "Access Adaptive and Thread-Aware Cache Partitioning in Multicore Systems," *Electronics*, vol. 7, no. 9, p. 172, 2018.

[4] Y. J. Kim and J. Kim, "ARC-H: Adaptive Replacement Cache Management for Heterogeneous Storage Devices," *Journal of Systems Architecture*, vol. 58, no. 2, pp. 109–118, 2012.

[5] N. Chaturvedi, J. P. Thoma, and S. Gurunarayanan, "Adaptive Zone-Aware Multi-bank On-Chip Last Level L2 Cache Partitioning for Chip Multiprocessors," *International Journal of Computer Applications*, vol. 7, no. 1, pp. 1–8, 2010.

[6] S. Srikantaiah, E. Kultursay, T. Zhang, M. Kandemir, M. J. Irwin, and Y. Xie, "MorphCache: A Reconfigurable Adaptive Multi-level Cache Hierarchy," in *Proc. Brazilian Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, 2012.

[7] G. Ayers, J. H. Ahn, C. Kozyrakis, and P. Ranganathan, "Memory Hierarchy for Web Search," in *Proc. International Symposium on High Performance Computer Architecture (HPCA)*, 2018.

[8] A. N. M. Imroz Choudhury and P. Rosen, "Abstract Visualization of Runtime Memory Behavior," in *IEEE International Working Conference on Software Visualization (VisSoft)*, 2011.

[9] J. W. Park, C. G. Kim, J. H. Lee, and S. D. Kim, "An Energy-Efficient Cache Memory Architecture for Embedded Systems," in *Proc. Embedded Systems Conference*, 2011.

[10] R. S. Shekhar and Y. N. Srikant, "An Adaptive, Energy Efficient Object Cache Architecture," Technical Report IISc-CSA-TR-2006-1, Indian Institute of Science, 2006.

[11] G. Ayers, J. Ahn, C. Kozyrakis, and P. Ranganathan, "Memory Hierarchy Design Considerations for Web Applications," *Google Research*, 2020.

[12] V. Ahire, P. Menon, A. Muley, and A. S. Prasad, "Random Adaptive Cache Placement Policy," *arXiv preprint arXiv:2502.02349*, 2025.

[13] J.-K. Peir, Y. Lee, and W. W. Hsu, "Capturing dynamic memory reference behavior with adaptive cache topology," in *Proceedings of the 8th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-VIII)*, pp. 134–143, 1998.

[14] T. R. Puzak, "Analysis of cache replacement algorithms," Ph.D. dissertation, Dept. of Electrical and Computer Engineering, Univ. of Massachusetts, Amherst, MA, 1985.

[15] A. Seznec, "A case for two-way skewed-associative caches," in *Proceedings of the 20th Annual International Symposium on Computer Architecture (ISCA)*, pp. 169–178, 1993.

[16] A. J. Smith, "Sequentiality and prefetching in database systems," *ACM Transactions on Database Systems*, vol. 3, no. 3, pp. 223–247, Sep. 1978.

[17] A. J. Smith, "Cache memories," *ACM Computing Surveys*, vol. 14, no. 4, pp. 473–530, 1982.

[18] D. Weiss, J. J. Wuu, and V. Chin, "The on-chip 3-Mb subarray-based third-level cache on an Itanium microprocessor," *IEEE Journal of Solid-State Circuits*, pp. 1523–1529, Nov. 2002.

[19] S. J. E. Wilton and N. P. Jouppi, "CACTI: An enhanced cache access and cycle time model," *IEEE Journal of Solid-State Circuits*, vol. 31, pp. 677–688, May 1996.